

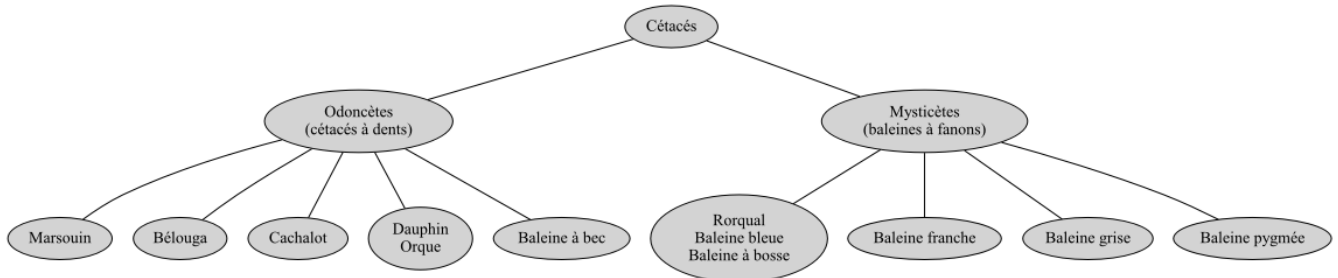


Le stockage de données dit « linéaire » en listes, en piles ou en files n'est pas adapté à toutes les données, en particulier aux structures hiérarchiques de données. Structures Hiérarchiques

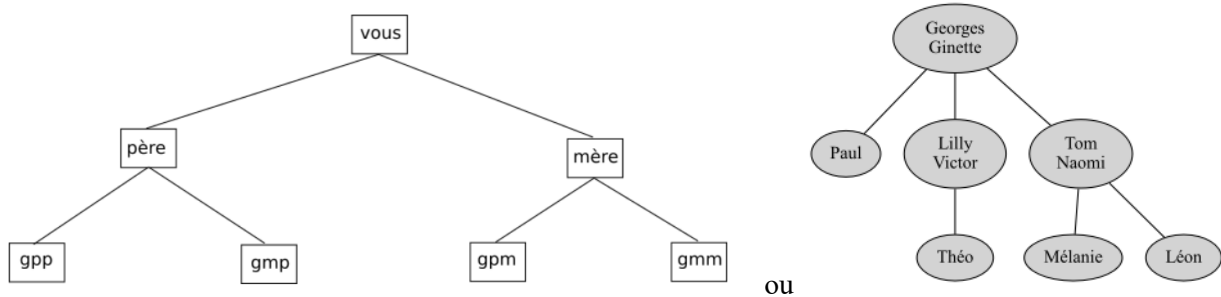
I. QUELQUES EXEMPLES

<https://www.rolandgarros.com/fr-fr/results/SM?round=5>

Ex 1 : Arbre

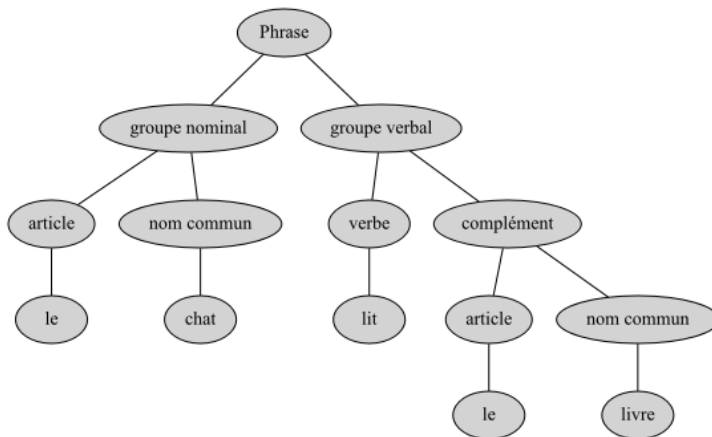


Ex 2 : Arbre

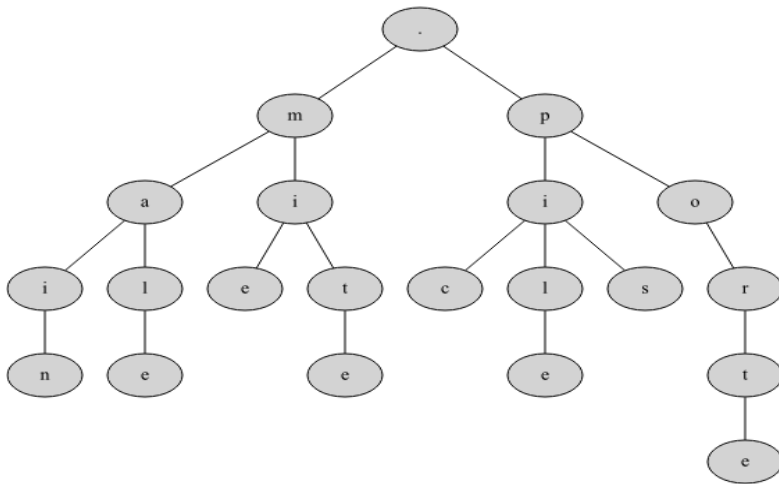


ou

Ex 3 : Arbre

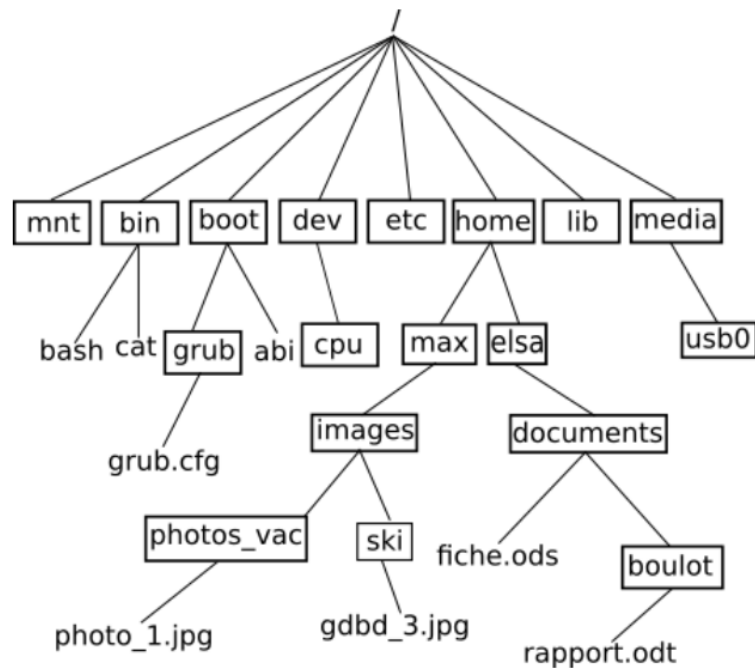


Ex 4 : Arbre



Exo 1. Rajouter les mots : malle et portail

Ex 5 : Arborescence



Exo 2. Créer un dossier pour la musique d'Elsa, et un dossier pour les photos de surf de Bixente

Ex 6 : Expressions algébriques

	$\left(\frac{y}{2} - t\right)(75 + z)$	$3 + \left(\frac{7}{3} - 1\right)^3$

Rq1. Les arbres sont étiquetés par des opérateurs, des constantes et des variables. La structure de l'arbre rend compte de la priorité des opérateurs et rend inutile tout parenthésage.

Ex 7 : D'autres exemples ?

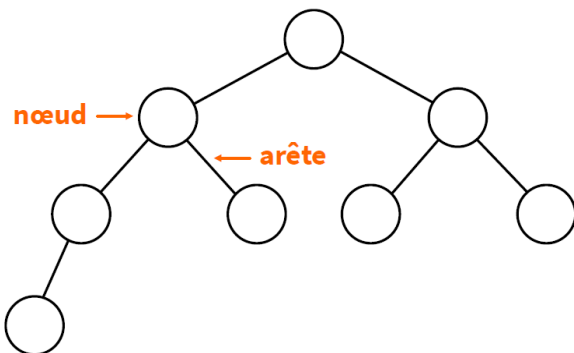
Les arbres sont des types abstraits très utilisés en informatique. On les utilise notamment quand on a besoin d'une structure hiérarchique des données par ex. :

- ✚ du plus générique au plus spécifique,
- ✚ du plus ancien au plus récent,
- ✚ du plus complexe au plus simple.



On parle d'arbres, d'arborescence ou de structures arborescentes

1. VOCABULAIRE



un **nœud** est caractérisé par une donnée, une **arête** (ou lien) relie deux nœuds.

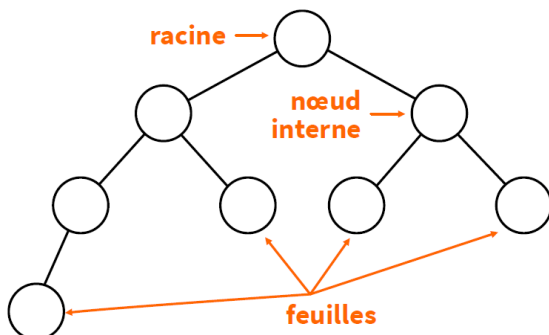
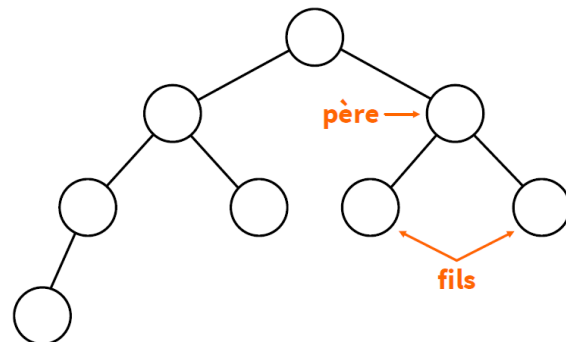
Rq2. L'arbre vide n'est pas un nœud.

les **fil**s sont l'ensemble des nœuds reliés à un même nœud par des arêtes entrantes.

Lorsque le nombre de fils des différents nœuds de l'arbre est inférieur ou égal à 2, on parle d'arbre **binaire**.

le **père** (ou parent) est le nœud relié à ses nœuds fils par une arête sortante.

un **sous-arbre** est l'ensemble des nœuds et arêtes d'un nœud parent et de ses fils.



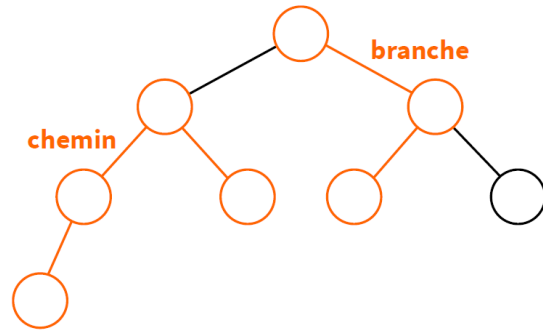
la **racine** est le seul nœud sans père.

une **feuille** est un nœud sans fils.

un **nœud interne** est un nœud qui n'est pas une feuille.

un **chemin** est une liste de nœuds reliés par des arêtes.

une **branche** est le chemin le plus court reliant un nœud à la racine.



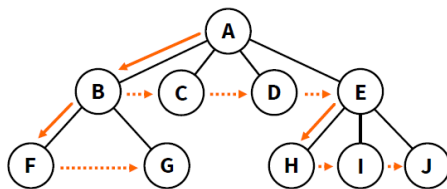
Exo 3. Tracer un arbre à 4 nœuds dont tous les nœuds internes n'ont qu'un fils. A quelle autre structure de données peut-on alors penser ?

2. ARBRE QUELCONQUE – ARBRE BINAIRE

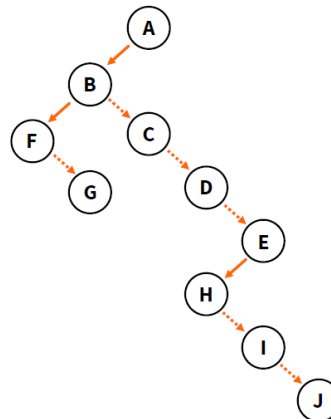


On choisit de particulariser les arbres binaires car il s'agit d'une forme très répandue et élémentaire et par ailleurs, tout arbre est représentable sous la forme d'un arbre binaire :

Arbre **n-aire**



Arbre **binaire** équivalent



II. ARBRES BINAIRES

1. DEFINITION RECURSIVE ET REPRESENTATION GRAPHIQUE

Un arbre binaire est un cas particulier de structure arborescente dans lequel chaque père a au plus deux fils.

Csq1. Si l'arbre est binaire, alors

soit il est vide (il ne contient aucun nœud) (on le notera Δ)

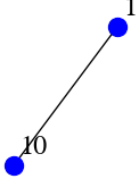
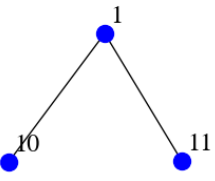
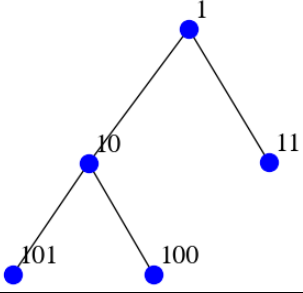
soit il est composé d'un nœud (sa racine), d'un sous-arbre gauche et d'un sous-arbre droit.

On le notera alors (v, g, d) où g et d sont eux même des arbres.

Rq3. La racine est donc reliée à la racine de chacun des sous-arbres.

Rq4. Comme pour les listes chaînées, cette notion apparaît donc naturellement récursive : un arbre binaire est construit à partir d'un élément et des sous-arbres binaires. Sa définition faisant apparaître la possibilité de l'arbre vide, elle fournit la condition d'arrêt de la récursivité.

Ex 8 : Si Δ désigne l'arbre vide :

définition	$(1, \Delta, \Delta)$	$(1, (10, \Delta, \Delta), \Delta)$	$(1, (10, \Delta, \Delta), (11, \Delta, \Delta))$	$(1, (10, (101, \Delta, \Delta), (100, \Delta, \Delta)), (11, \Delta, \Delta))$
Représentation graphique				

Exo 4. Représenter graphiquement chacun des arbres définis ci-dessous :

- $(3, (1, \Delta, (4, (1, \Delta, (5, \Delta, \Delta))), \Delta), \Delta)$
- $(3, (1, (1, \Delta, \Delta), \Delta), (4, (5, \Delta, \Delta), (9, \Delta, \Delta)))$
- $(3, (1, (1, \Delta, \Delta), (5, \Delta, \Delta)), (4, (9, \Delta, \Delta), (2, \Delta, \Delta)))$

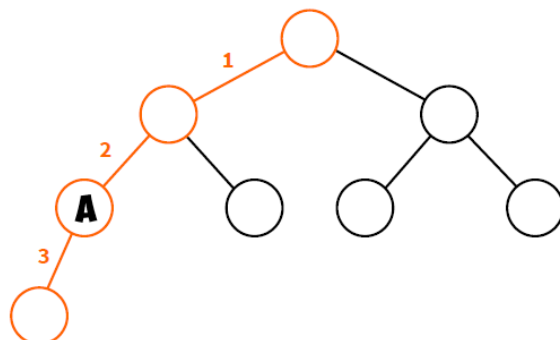
2. MESURES DES ARBRES BINAIRES

la **taille** d'un arbre est le nombre de nœuds de l'arbre.

la **profondeur** d'un nœud est le nombre d'arêtes sur la branche qui le relie à la racine.

la **hauteur** d'un arbre est la profondeur maximale de l'ensemble des nœuds de l'arbre.

Ex 9 : Dans le cas suivant :



taille =

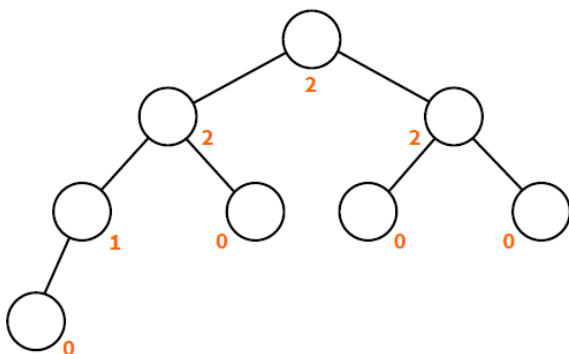
profondeur de A =

hauteur =

L'arité ou degré d'un nœud est le nombre de fils du nœud.

L'arité ou degré d'un arbre est le nombre maximal de fils des nœuds de l'arbre.

Ex 10 :



Rq5. Un arbre binaire est donc un arbre d'arité deux.

Exo 5. Pour chacun des arbres déjà représentés ci-dessous, déterminer sa taille et sa hauteur, le nombre de feuilles, le nombre de nœuds à chaque profondeur :

- a. Δ
- b. $(3, (1, \Delta, (4, (1, \Delta, (5, \Delta, \Delta)), \Delta)), \Delta)$
- c. $(3, (1, (1, \Delta, \Delta), \Delta), (4, (5, \Delta, \Delta), (9, \Delta, \Delta)))$
- d. $(3, (1, (1, \Delta, \Delta), (5, \Delta, \Delta)), (4, (9, \Delta, \Delta), (2, \Delta, \Delta)))$

Exo 6. Dessiner tous les arbres binaires ayant respectivement 3 et 4 nœuds.

Exo 7. Sachant qu'il y a 1 arbre binaire vide, 1 arbre binaire à 1 nœud, 2 arbres binaires à 2 nœuds, 5 arbres binaires à 3 nœuds et 14 arbres binaires à 4 nœuds, calculer le nombre d'arbres binaires à 5 nœuds. (Ne pas les construire tous... Juste les dénombrer.)

Exo 8. Ranger les mots BON, BONJOUR, BORD, BOND, BOREALE, BIEN dans une structure d'arbre lexicographique.

III. IMPLEMENTATION EN PYTHON



Il y a de nombreuses implémentations possibles. On en présente une, analogue à celle des listes chaînées.

1. CLASSE NŒUD



[classe_noeud.py](#)



De manière analogue à la classe Maillon des listes, on peut utiliser une classe décrivant les nœuds d'un arbre : chaque nœud de l'arbre est donc une instance de la classe Nœud, et l'arbre vide est représenté par **None**.

```
class Noeud:
    #attribut de classe
    arbre_vide = None

    """Constructeur d'un noeud d'un arbre binaire"""
    def __init__(self, v, g, d):
        self._valeur = v
        self._gauche = g
        self._droit = d

    #selecteurs
    def valeur(self):
        return self._valeur

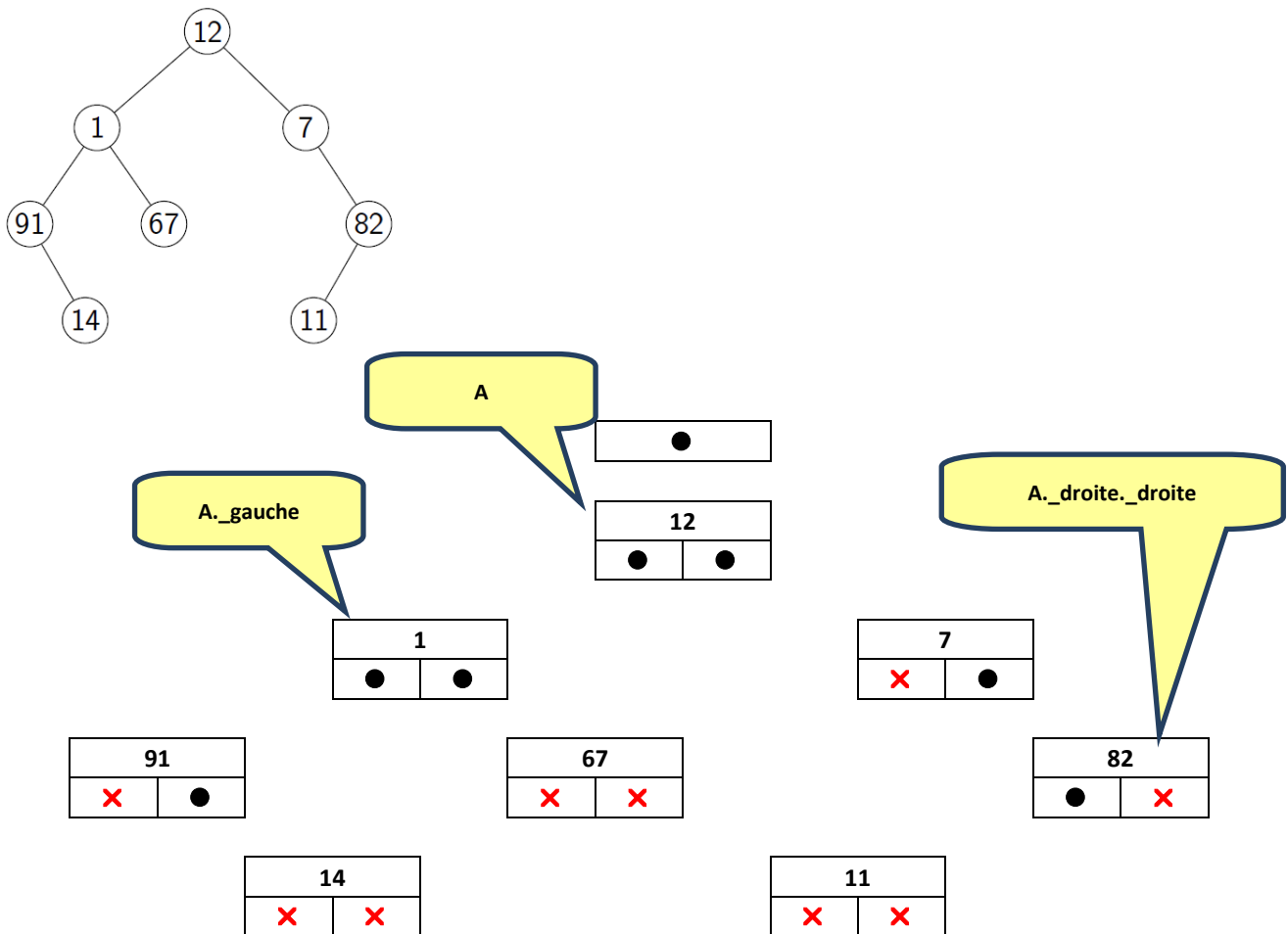
    def gauche(self):
        return self._gauche
    def droit(self):
        return self._droit

    #methode de classe
    def est_vide(arbre):
        return arbre is Noeud.arbre_vide
```

Rq6. On pourrait rajouter aux sélecteurs (*getter* en anglais) qui permettent d'obtenir la valeur associée à un nœud des accesseurs (*setter* en anglais) qui permettent d'affecter une valeur à un nœud.

2. ARBRE BINAIRE

Ex 11 : On visualise l'organisation en mémoire de l'arbre A suivant :



Exo 9.

1. Construire, en *python* et en utilisant la classe Nœud, l'arbre feuille **A_g** suivant :

« D »

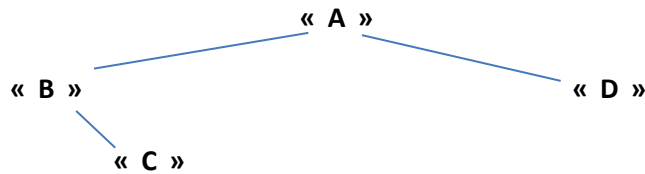
2. Construire, en *python* et en utilisant la classe Nœud, l'arbre **A_1** suivant :

« A »
« B »

3. Construire, en utilisant la classe Nœud, l'arbre **A_2** suivant :

« A »
« B » « D »

4. Construire, en utilisant la classe `Nœud`, l'arbre **A_3** suivant :



Définition récursive :

Un arbre est donc soit la valeur **None**, soit un objet de la classe **Nœud** dont l'attribut **_valeur** contient la valeur de sa racine, et les attributs **_gauche** et **_droite** contiennent un arbre.

Rq 7. Comme pour les listes d'autres représentations de la classe **Nœud** sont possibles :

On pourrait représenter un Nœud

par un triplet, et dans ce cas la construction de l'arbre **A_2** devient :

`L_2 = ('A', ('B', None, None), ('D', None, None))`

par un tableau de taille 3, et dans ce cas la construction de l'arbre **A_2** devient :

`L_2 = ['A', ['B', None, None], ['D', None, None]]`

Mais l'utilisation des attributs valeur, gauche et droite rend le code plus lisible

3. QUELQUES OPERATIONS SUR LES ARBRES

Exo 10. A partir de la classe **Nœud**, implémenter les fonctions suivantes dont on donne les *docstrings* (à reporter dans le code) :

- a. Fonction **est_feuille(A)** :

à partir d'un arbre binaire A, renvoie un booléen indiquant si A est une feuille

- b. Fonction **compte_feuilles(A)** :

à partir d'un arbre binaire A, renvoie un entier indiquant le nombre de feuilles de A (penser à la récursivité naturelle de la définition de l'arbre)

$$\text{compte_feuilles}(A) = \begin{cases} 0 & \text{si} \\ 1 & \text{si} \end{cases} \quad \text{sinon}$$

- c. Avec une simple modification, calculer la taille d'un arbre, c'est-à-dire son nombre de nœuds avec la fonction **taille(A)** (penser à la récursivité naturelle de la définition de l'arbre)

$$\text{taille}(A) = \begin{cases} 0 & \text{si} \\ \text{sinon} \end{cases}$$

à partir d'un arbre binaire A, renvoie un entier indiquant la taille de A

- d. On souhaite enfin calculer la hauteur d'un arbre, c'est-à-dire sa profondeur maximale :

fonction **hauteur(A)**

à partir d'un arbre binaire A, renvoie un entier indiquant la hauteur de A (**supposé non vide**)

$$hauteur(A) = \begin{cases} 0 \\ \end{cases}$$

si ,
si $A.droit()$ est vide ,
si ,
sinon.



[arbre_operations.py](#)

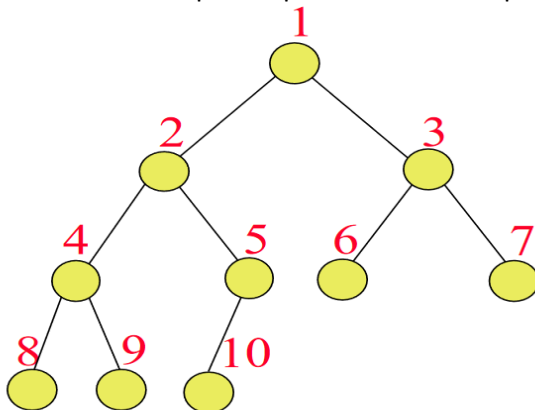
4. PARCOURS DES ARBRES BINAIRES



[parcours.py](#)

Exo 11. Parcours infixe

- A partir de la fonction **taille()** écrite récursivement à l'exercice précédent, écrire une fonction **parcours_infixe()** qui affiche les valeurs de chaque nœud exploré séparées par des espaces.
- Noter le chemin emprunté par cette fonction pour l'arbre A ci-dessous :



Parcours_infixe :

le parcours infixe est un **parcours en profondeur** :

en partant de la racine, il descend **au fond de l'arbre** par le chemin le plus à gauche puis tente de remonter par la droite en visitant chaque sommet **dès qu'il passe en-dessous**.

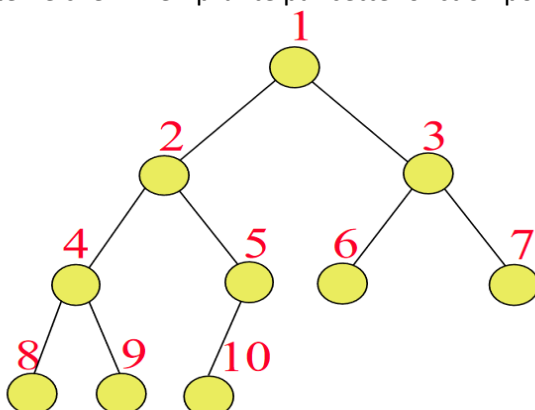
Exo 12. Parcours préfixe

- écrire une fonction **parcours_prefixe()** qui affiche les valeurs de chaque nœud exploré séparées par des espaces.

le parcours préfixe est un **parcours en profondeur** :

en partant de la racine, il descend **au fond de l'arbre** par le chemin le plus à gauche puis tente de remonter par la droite en visitant chaque sommet **dès qu'il passe à gauche** (donc **avant** d'aller plus loin et dès qu'il le peut)

- Noter le chemin emprunté par cette fonction pour l'arbre A ci-dessous :



Parcours_prefixe :

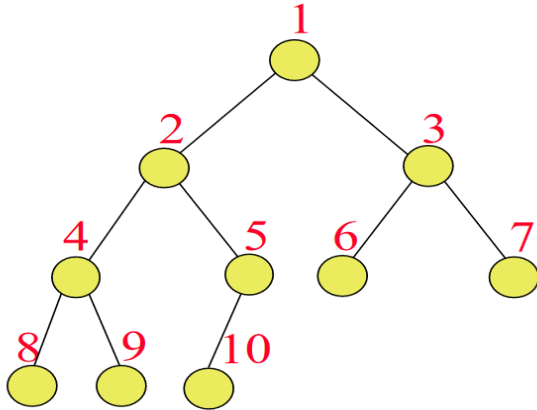
Exo 13. Parcours suffixe ou postfixe

- a. écrire une fonction `parcours_postfixe()` qui affiche les valeurs de chaque nœud exploré séparées par des espaces.

le parcours suffixe ou postfixe est un **parcours en profondeur** :

en partant de la racine, il descend **au fond de l'arbre** par le chemin le plus à gauche puis tente de remonter par la droite en visitant chaque sommet quand **il passe à droite** (c'est-à-dire **à sa dernière occasion**)

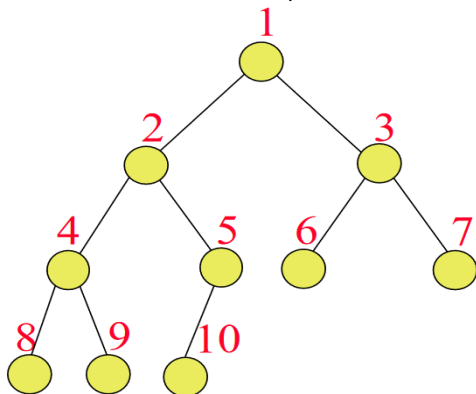
- b. Noter le chemin emprunté par cette fonction pour l'arbre A ci-dessous :



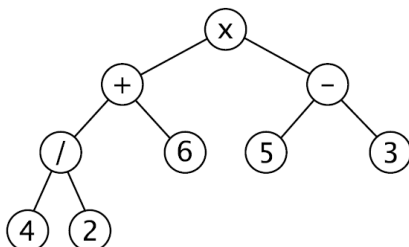
Parcours_postfixe :

Exo 14. Parcours en largeur :

- a. Importer la fonction `hauteur()`
 b. écrire une fonction `parcours_en_largeur()` qui prend en paramètre l'arbre A ci-dessous, le parcourt et affiche : `1 2 3 4 5 6 7 8 9 10`. (...sans utiliser de boucle du style `for i in range(1,10) : print(i,end= " ")...`)

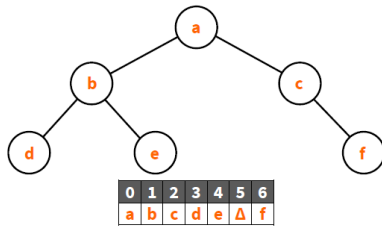
5. **Exos**

Exo 15. Quel est le type de parcours qui permettra d'évaluer l'expression algébrique rangée dans cet arbre :



Exo 16. On veut représenter un arbre dans un tableau :
LISTE UNIQUE (MÉTHODE D'EYTZINGER)

Liste dans laquelle les fils gauche et droit d'un nœud i sont rangés respectivement dans les cases $2i+1$ et $2i+2$.



Stockage mieux adapté aux **arbres complets**.

- Proposer une fonction **table(a)** qui prend un **arbre a** en paramètre et qui renvoie le tableau correspondant (sous forme de liste Python) pcha
- Ecrire une fonction qui affiche l'arbre **a** à la console dans le format suivant :
 par ex. avec le tableau ci-dessus et en formatant la valeur d'un nœud sur 2 caractères :

```

-----a-----
---b---          ---c---
-d -      -e -      -f -
  
```

- Tester sur l'arbre A des exos précédents ayant permis l'exploration des différents parcours.
 réponse console attendue (on formate la valeur d'un nœud sur 2 caractères) :

```

----- 1 -----
----- 2 -----          ----- 3 -----
--- 4 ---          --- 5 ---          --- 6 ---          --- 7 ---
- 8 -      - 9 -      -10-
  
```

- Ecrire une fonction **graphique(a)** qui affiche l'arbre **a** à l'aide du module **matplotlib**.
- Tester sur l'arbre A des exos précédents ayant permis l'exploration des différents parcours.

Exo 17. Coder la fonction suivante dont on donne la docstring :

```

def table_infixe(a, tab=None):
    """renvoie la liste des éléments de a lus dans un parcours infixe"""
    if a is None:
        return []
    if tab==None: tab=[]
    ....
  
```

Exo 18. [TD Expression Algébrique.docx](#)

IV. ARBRES BINAIRES DE RECHERCHE (ABR)



<https://www.lumni.fr/video/arbres-binaires-de-recherche>

1. ABR ET PARCOURS CROISSANT

Un **arbre binaire de recherche** (ABR) est un arbre binaire respectant la propriété suivante : pour chaque nœud de valeur v , les nœuds du sous-arbre gauche ont une valeur strictement inférieure à v et ceux du sous-arbre droit ont une valeur strictement supérieure à v .

Exo 19. Parcours croissant dans un ABR

- a. Tracer un ABR de racine 15, dont les nœuds portent les valeurs suivantes :
10-19-17-13-16-12-14-8
- b. Tracer un ABR de racine 15, de hauteur 3, dont les nœuds portent les mêmes valeurs :
10-19-17-13-16-12-14-8

- c. Quel est le parcours qui dans tous les cas ordonne les nœuds par valeurs croissantes ?

2. IMPLEMENTATION EN PYTHON



Un ABR profite des propriétés des arbres binaires. On peut le représenter en Python exactement comme on l'a fait pour les arbres binaires avec la classe Nœud à condition de respecter deux nouvelles contraintes : on doit pouvoir comparer les valeurs des nœuds avec les opérateurs $<$ ou $>$ et on doit respecter la contrainte d'ordre imposée à tous les ABR. Les fonctions définies sur les arbres binaires (**taille()**, **hauteur()**, **parcours_xxx()**,...) restent valables sur les ABR.



[ABR_construction.py](#)

a) Ajouter une valeur dans un ABR

Exo 20. Dans l'arbre précédent, ajouter à la main les valeurs 6, puis 18, puis 20.



[ABR_construction.py](#)

Exo 21.

- a. En profitant de la définition récursive d'un arbre, compléter la formulation récursive de la fonction **ajouter(x,a)** qui renvoie l'arbre **a** auquel on a ajouté la valeur **x** :

$$ajouter(x,a) = \begin{cases} & \text{si } a \text{ est l'arbre vide} \\ & \text{si l'arbre est non vide et } x < a.valeur() \\ & \text{si} \end{cases}$$

- b. Dans un fichier **ABR.py** qui importe la classe **nœuds**, implémenter la fonction **ajouter(x,a)**
- ```
def ajouter(x, a):
 """ajoute x à l'ABR a, renvoie un nouvel ABR"""
```

### Exo 22. Construire un ABR

- c. Implémenter la fonction suivante :
- ```
def construire(valeurs):
    """renvoie l'ABR constitué des valeurs prises successivement
       dans la liste valeurs.
       valeurs[0] est la racine"""
```
- d. Tester avec la liste **L0=[6,8,3,1,4,9,2,7,5]**, en entrée, afficher le parcours infixe
- e. Tester avec la liste **L=[15,10,19,17,13,16,12,8,14]** en entrée, afficher le parcours infixe, puis la hauteur et le parcours en largeur.
- f. Faire de même avec un **random.shuffle()** de la liste **L**.
- g. Faire de même avec un **L.sort()** liste triée d'éléments de **L**
- h. Commenter.

b) Rechercher une valeur dans un ABR

- a. En profitant de la définition récursive d'un arbre, compléter la formulation récursive de la fonction **appartient(x,a)** qui renvoie un booléen précisant si la valeur **x** appartient à l'arbre **a**:

$$appartient(x,a) = \begin{cases} & \text{si } a \text{ est l'arbre vide} \\ & \text{si l'arbre est non vide et } x = a.valeur() \\ & \text{si l'arbre est non vide et } x < a.valeur() \\ & \text{si} \end{cases}$$

- b. Dans le fichier **ABR.py** qui importe la classe **nœuds**, implémenter la fonction **appartient(x,a)**

```
def appartient(x, a):
    """renvoie un booléen précisant si x apparait dans l'ABR a"""
```

Exo 23. Une bibliothèque contient des livres répartis dans 17576 salles reliées les unes aux autres par des portes. Chaque salle conduit à deux autres salles et chaque salle regroupe les livres dont le titre commence par un même « mot » de trois lettres.
 Dans la première salle, on a rangé les livres dont le titre commence par MDB.
 Si le titre commence par trois lettres précédant MDB dans l'ordre alphabétique, il faut prendre la porte de gauche, sinon la porte de droite.
 Si l'on a pris la porte de gauche, on trouve dans cette salle les titres commençant par GBN. En prenant la porte de gauche on cherchera les titres précédant GBN dans l'ordre alphabétique, et inversement en prenant la porte de droite.
 Pourquoi y-t-il 17576 salles ?
 Combien de salles doit-on traverser au maximum pour trouver un livre ?

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z

Exo 24. Dans un ABR de 255 nœuds, combien de nœuds faut-il explorer pour trouver un nœud en particulier ? (Envisager plusieurs cas, les meilleurs et les pires).
 et 2047 nœuds ?
 et $2^p - 1$ nœuds ?
 et N nœuds si $2^p \leq N \leq 2^{p+1} - 1$ (ici $p \leq \log_2(N) < p+1$)

Un tel algorithme de recherche dans un ABR est de complexité logarithmique lorsque l'arbre est équilibré

Rq8. pour tout arbre de hauteur h et de taille $n \geq 2$, on a : $h+1 \leq n \leq 2^{h+1} - 1$



Si l'arbre est linéaire, cas de la plus grande hauteur $h_{\max}$ possible on vérifie $h_{\max} + 1 = n$

Dans le cas général, pour tout ABR : $h+1 \leq h_{\max} + 1 \leq n$

Si l'arbre est complet (cas de deux fils exactement par nœud, sauf pour les feuilles), c'est un cas de hauteur minimale $h_{\min}$; on peut alors écrire que n est la somme des nombres de nœuds à chaque niveau, soit :

$$n = 1 + 2^1 + 2^2 + \dots + 2^{h_{\min}} = \frac{1 - 2^{h_{\min} + 1}}{1 - 2} = 2^{h_{\min} + 1} - 1$$

Dans le cas général, pour tout ABR, puisque $h_{\min} \leq h$: $n = 2^{h_{\min} + 1} - 1 \leq 2^{h+1} - 1$

3. Exos

Exo 25. On utilise une structure d'ABR, **A_Animaux**, pour rechercher une valeur dans la liste :

L_Animaux=['chat','chien','souris','araignée','crapaud','grenouille','lézard','zèbre'] :

- a. Vérifier que l'éléphant n'est pas dans l'arbre mais que le zèbre y est :
appartient("éléphant",A_animaux) renvoie **False**
appartient("zèbre",A_animaux) renvoie **True**
- b. Ecrire une fonction qui renvoie le premier élément dans l'ordre alphabétique (sans trier toute la liste et en utilisant l'ABR)

```
def premier(abr):  
    """renvoie le premier dans l'ordre de l'ABR"""
```

- c. Ecrire une fonction qui affiche la liste dans l'ordre alphabétique.

Exo 26. Écrire une fonction **trier_par_ABR(tab)** qui utilise une structure d'ABR pour trier un tableau d'entiers (la fonction ne renvoie rien mais modifie la liste passée en argument)

Exo 27. [_TD ABR Villes\TD ABR Villes.docx](#)